

# ClassRoomSTORM Studio V1.1

User Help — Virtual and Experimental Super-Resolution Microscopy

Awanish Pratap Singh

Software version 1.1

## Abstract

This Help document explains how to install and use ClassRoomSTORM Studio V1.1, a desktop teaching application for localization-based super-resolution microscopy. It is task-oriented: it tells you what each screen and control does. The physics behind the method — diffraction, blinking, localization, and precision — is explained in the companion document, *ClassRoomSTORM Theory Notes*.

## Contents

|           |                                                                 |           |
|-----------|-----------------------------------------------------------------|-----------|
| <b>1</b>  | <b>What ClassRoomSTORM Is and Is Not</b>                        | <b>3</b>  |
| <b>2</b>  | <b>Concepts in 60 Seconds</b>                                   | <b>3</b>  |
| <b>3</b>  | <b>Installation and Setup</b>                                   | <b>3</b>  |
| <b>4</b>  | <b>Starting ClassRoomSTORM Studio</b>                           | <b>4</b>  |
| <b>5</b>  | <b>Quick Start: First Virtual Experiment and Reconstruction</b> | <b>5</b>  |
| <b>6</b>  | <b>Virtual Experiment Step by Step</b>                          | <b>6</b>  |
| 6.1       | Step 1 — Pattern . . . . .                                      | 6         |
| 6.2       | Step 2 — Blinking . . . . .                                     | 6         |
| 6.3       | Step 3 — Image Formation . . . . .                              | 9         |
| 6.4       | Step 4 — Preview . . . . .                                      | 9         |
| 6.5       | Step 5 — Export . . . . .                                       | 10        |
| <b>7</b>  | <b>SR Reconstruction Step by Step</b>                           | <b>10</b> |
| 7.1       | Step 1 — Load Video . . . . .                                   | 10        |
| 7.2       | Step 2 — Calibration . . . . .                                  | 10        |
| 7.3       | Step 3 — Preprocess . . . . .                                   | 10        |
| 7.4       | Step 4 — Blinker Mode . . . . .                                 | 11        |
| 7.5       | Step 5 — Processing . . . . .                                   | 11        |
| 7.6       | Step 6 — Run and Results . . . . .                              | 12        |
| <b>8</b>  | <b>Linked Profile Viewer</b>                                    | <b>12</b> |
| <b>9</b>  | <b>Reading Output Files and Diagnostic Figures</b>              | <b>14</b> |
| <b>10</b> | <b>Input and Output File Table</b>                              | <b>14</b> |
| <b>11</b> | <b>Troubleshooting</b>                                          | <b>14</b> |
| <b>12</b> | <b>Glossary</b>                                                 | <b>16</b> |



# 1 What ClassroomSTORM Is and Is Not

ClassroomSTORM Studio V1.1 is a desktop teaching application for *localization-based super-resolution microscopy*. It has two workflows:

- the **Virtual Experiment**, which creates synthetic “blinking” videos from a pattern of points; and
- **SR Reconstruction**, which turns a blinking video into a sharper *reconstructed image* by locating individual bright spots frame by frame and accumulating their positions.

ClassroomSTORM reproduces the *statistical and computational logic* of single-molecule localization microscopy (SMLM) — the family of methods that includes STORM and PALM — in a form that needs no lasers, no fluorescence optics, and no scientific camera.

ClassroomSTORM is a *learning tool*, not a fluorescence microscope and not a research-grade analysis package. Its reconstruction deliberately uses a simple, fully transparent method — detect bright spots, take an intensity-weighted centroid, and accumulate — so that every step is visible and explainable. It does not perform Gaussian model fitting, maximum-likelihood estimation, drift correction, or applied physical calibration; those belong to the theory background and to possible future extensions (see the *Theory* document).

## 2 Concepts in 60 Seconds

### Blinking video.

A video with many frames in which, at any instant, only a few light sources (“blinkers”) are switched on. Each blinker appears as a small blurred spot.

### Localize.

In each frame, ClassroomSTORM finds the centre of every bright spot. Although the spot is blurry and several pixels wide, its *centre* can be estimated to better than one pixel.

### Accumulate.

The centres found in all frames are collected into a single image — the *reconstructed image*. Because it is built from sharp centres rather than blurry spots, it is much crisper than the raw video.

### Compare.

The plain average of the video frames (**Raw Mean**) is blurry; the **Super-Resolution** image is the accumulated reconstruction. Comparing the two is the central activity.

When you create a Virtual Experiment, ClassroomSTORM also saves the exact positions it used, in a file called `truth.csv`. Reconstruction does *not* read this file. You compare your reconstruction against it only *afterwards*, to score your result — like not peeking at an answer key. Keep `truth.csv` in its original folder and do not feed it into the reconstruction.

## 3 Installation and Setup

ClassroomSTORM V1.1 is a Python desktop application. The user package includes ready-made setup and launch scripts, so most users never need to type any commands.

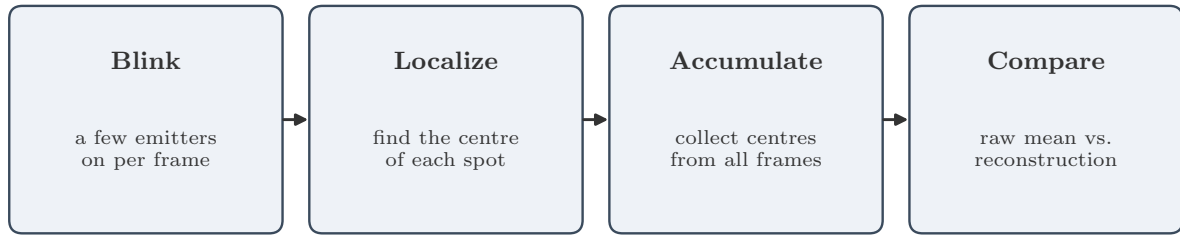


Figure 1: The ClassroomSTORM loop: blink, localize, accumulate, and compare.

**Requirements.** The Windows setup script looks for Python 3.11, 3.12, or 3.13 automatically, using the Python launcher (`py`) when available. If no supported Python is found, it can offer to install Python 3.11 with `winget` or open the official Python download page. On macOS, install Python 3.10 or newer from <https://www.python.org/> before running setup. First-time setup downloads roughly 250–350 MB of packages from the Python Package Index (PyPI), so an internet connection is needed — unless your package includes an offline `wheels` folder (see below).

#### Recommended — run the setup script.

- **Windows:** double-click `setup_windows.bat`.
- **macOS:** double-click `setup_macos.command`. The first time, macOS may block it; right-click the file and choose **Open**, or run `chmod +x setup_macos.command` in Terminal once.

The script builds a private environment (a `.venv` folder inside the package) and installs the required packages. When it finishes it prints *Setup completed*. If it fails partway it removes the incomplete environment, so the launcher never starts from a broken install.

**Offline installation.** If the package folder contains a `wheels` folder, the setup script installs from it automatically and no internet access is needed. This is useful on restricted or proxy-protected networks.

**Manual installation (advanced).** Instead of the script you may open a terminal in the package folder and run `python -m venv .venv`, activate it (Windows PowerShell: `.\.venv\Scripts\Activate.ps1`, macOS or Linux: `source .venv/bin/activate`), then run `python -m pip install --upgrade pip` followed by `python -m pip install -r requirements.txt`. The standard packages are `numpy`, `opencv-python`, `matplotlib`, `Pillow`, and `PySide6`.

**Graphviz** produces cleaner pipeline diagrams. Without it, ClassroomSTORM draws the same diagrams with Matplotlib instead, so it is not required. **NVIDIA CUDA GPU** acceleration is experimental; install it with `python -m pip install -r requirements-gpu.txt` only if you have a CUDA-capable NVIDIA card. AMD, Intel, and Apple-Silicon GPUs are not supported in V1.1 — those systems use the CPU.

## 4 Starting ClassroomSTORM Studio

Start the application with the launch script in the package folder: **Windows** — double-click `launch_studio_windows.bat`; **macOS** — double-click `launch_studio_macos.command`. The

launch script checks that the required packages are installed and, if they are missing, asks you to run the setup script first. Advanced users may instead run `python apps/ClassRoomSTORM_Studio.py` from an activated environment.

The ClassroomSTORM Studio launcher window offers two large buttons:

- **Virtual Experiment** — create blinking data from a logo, letter, or pattern (section 6);
- **SR Reconstruction** — reconstruct a recorded or simulated blinking video (section 7).

Two text links, **Help** and **Theory**, open this document and the Theory companion (the PDF files in the `docs` folder).

When you click **Virtual Experiment** or **SR Reconstruction**, the launcher window fades out and *closes itself*; the chosen workflow then opens as a separate window. This is normal — the application has not crashed. To return to the launcher, use the **Back to Studio** toolbar button at the top of the workflow window, or run the launch command again.

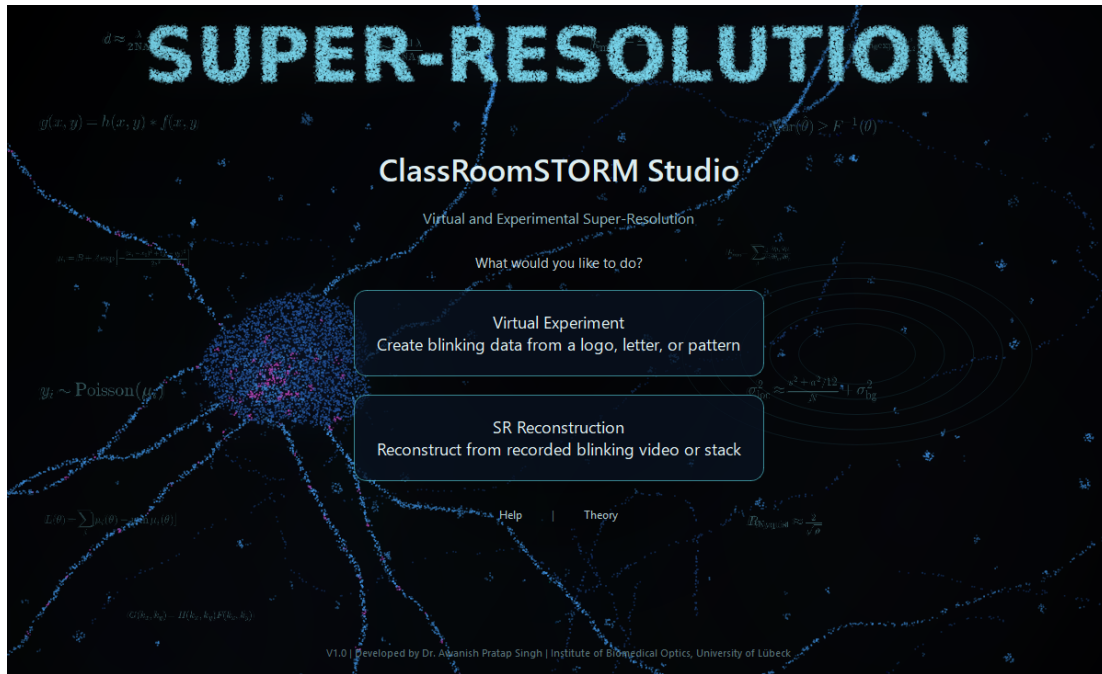


Figure 2: The ClassroomSTORM Studio launcher: the two workflow buttons (**Virtual Experiment**, **SR Reconstruction**) and the **Help** and **Theory** links, over the animated localization background.

## 5 Quick Start: First Virtual Experiment and Reconstruction

This walkthrough takes about ten minutes and uses only simulated data.

1. **Launch** the Studio and click **Virtual Experiment**.
2. On the **1 Pattern** page, set **Pattern** to **text** and type a short word (for example ABBE) in the **Text** box.
3. On the **2 Blinking** page, set **Frames** to about 300 for a quick first run.
4. On the **3 Image Formation** page, leave the defaults.

5. On the **4 Preview** page, click **Preview Source Mask** and then **Preview Sample Blink Frame** to see what you are about to generate.
6. On the **5 Export** page, click **Export Dataset**. Note the output folder shown; it now contains `blinking_video.mp4`.
7. **Relaunch** the Studio (it closed in step 1) and click **SR Reconstruction**.
8. On the **1 Load Video** page, click **Browse Video** and select the `blinking_video.mp4` you just created. The first frame appears.
9. Leave **Blinker mode** on **auto** and **Processing backend** on **Serial**. On the **6 Results** page, click **Run Reconstruction**.
10. Open **Main Results**, then the **Linked Profiles** tab. Click on a bright part of the image and compare the wide red curve (raw) with the narrow blue curve (reconstruction).

## 6 Virtual Experiment Step by Step

The Virtual Experiment builds a synthetic blinking video. Its window has five numbered steps in the left-hand list: **1 Pattern**, **2 Blinking**, **3 Image Formation**, **4 Preview**, and **5 Export**.

### 6.1 Step 1 — Pattern

The pattern defines *where* the source points sit. Choose one of three pattern types with the **Pattern** control.

| Control                       | What it does                                                                                                                                | Default             |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| <b>Pattern</b>                | Source pattern type: <code>text</code> , <code>dots</code> , or <code>logo</code> .                                                         | <code>text</code>   |
| <b>Text</b>                   | The word drawn in <code>text</code> mode, using a fixed 16×16 LED-style alphabet (A–Z, 0–9, space, -, _).                                   | <code>ABBE</code>   |
| <b>Dot mode</b>               | For <code>dots</code> mode: <code>single</code> (one point), <code>pair</code> (two close points), or <code>random</code> (several points). | <code>single</code> |
| <b>Logo</b>                   | For <code>logo</code> mode: an image file (use <b>Browse Logo</b> ). The image is converted to a black-and-white mask.                      | —                   |
| <b>Logo scale</b>             | Scales the logo mask (0.1–5.0).                                                                                                             | <code>1.0</code>    |
| <b>Logo sampling</b>          | How logo pixels become source points: <code>grid</code> , <code>all_pixels</code> , or <code>random_max_points</code> .                     | <code>grid</code>   |
| <b>Logo grid step px</b>      | Spacing of the sampling grid, in pixels (1–100).                                                                                            | <code>1</code>      |
| <b>Logo max source points</b> | Upper limit on logo points; 0 means no limit.                                                                                               | <code>0</code>      |

Table 1: Controls on the **1 Pattern** page.

### 6.2 Step 2 — Blinking

This page controls *how the source points switch on and off* over the frames of the video.

If too many blinkers are on at once and they sit close together, their blurred spots overlap and cannot be told apart. Sparse, well-separated blinking is what makes localization possible — this is the core idea of the method (see the *Theory* document).

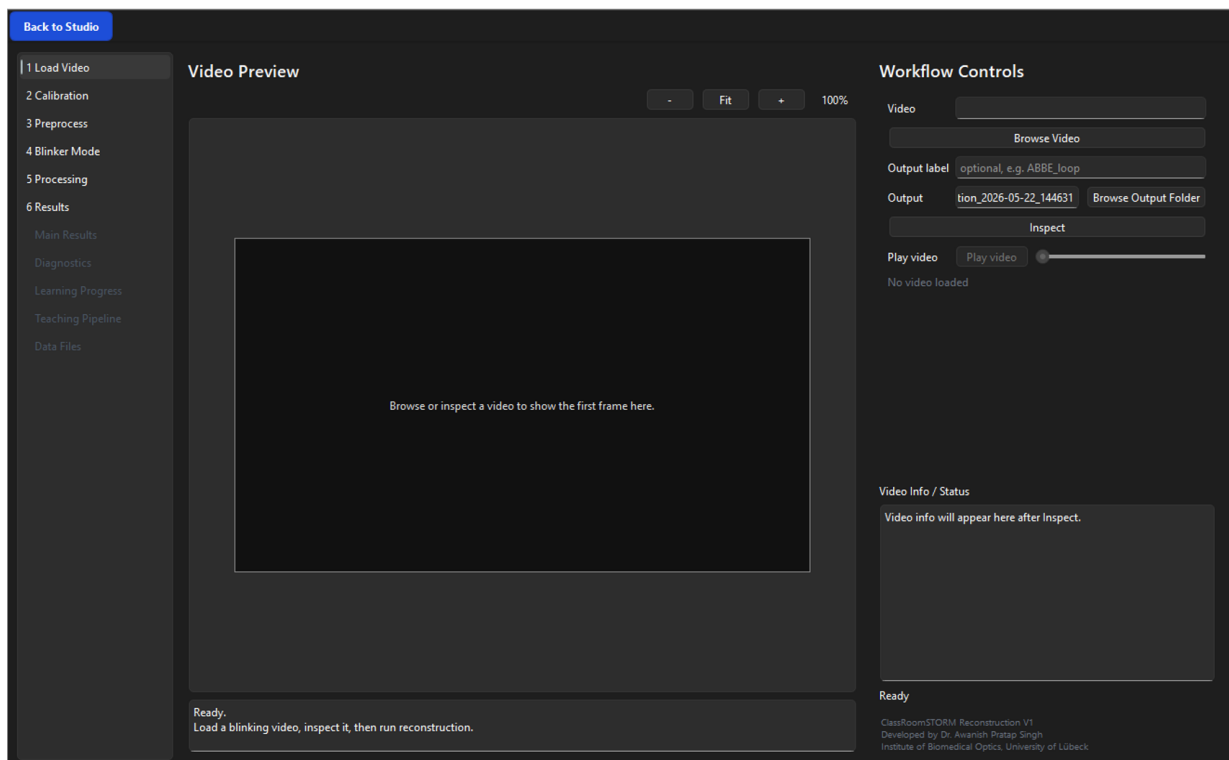
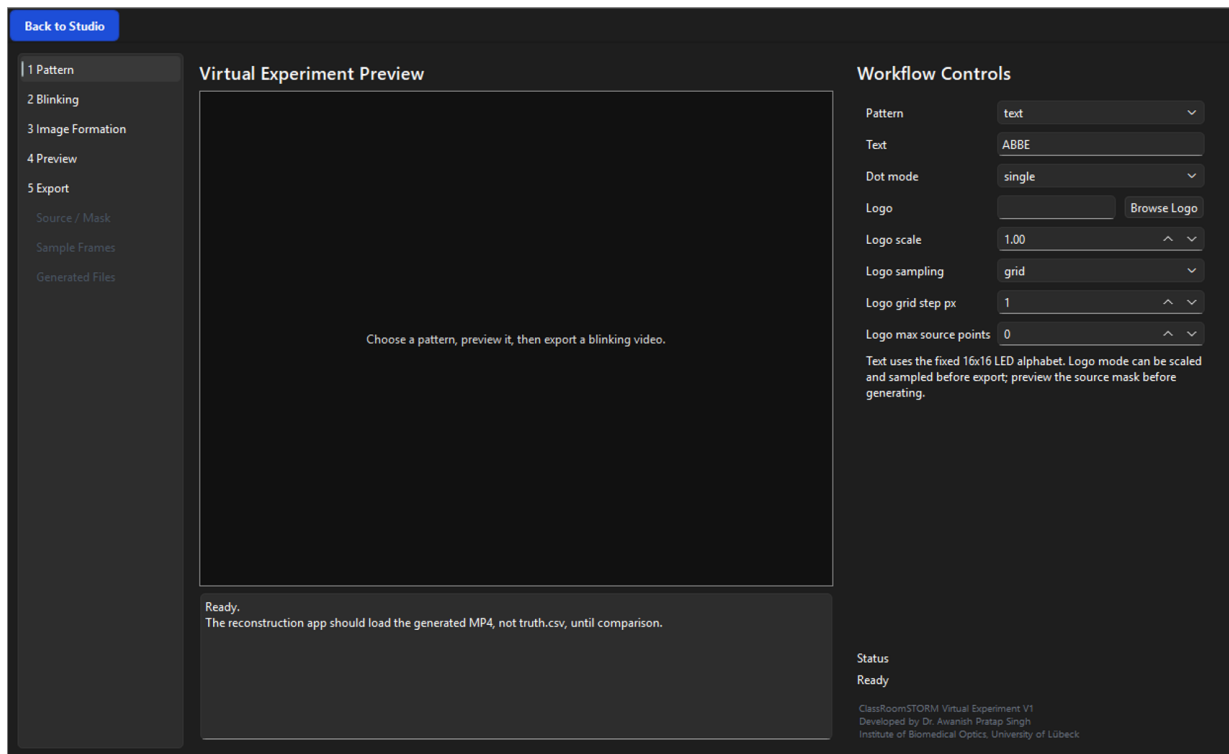


Figure 3: The two workflow windows used in the Quick Start: the Virtual Experiment (top) and SR Reconstruction (bottom).

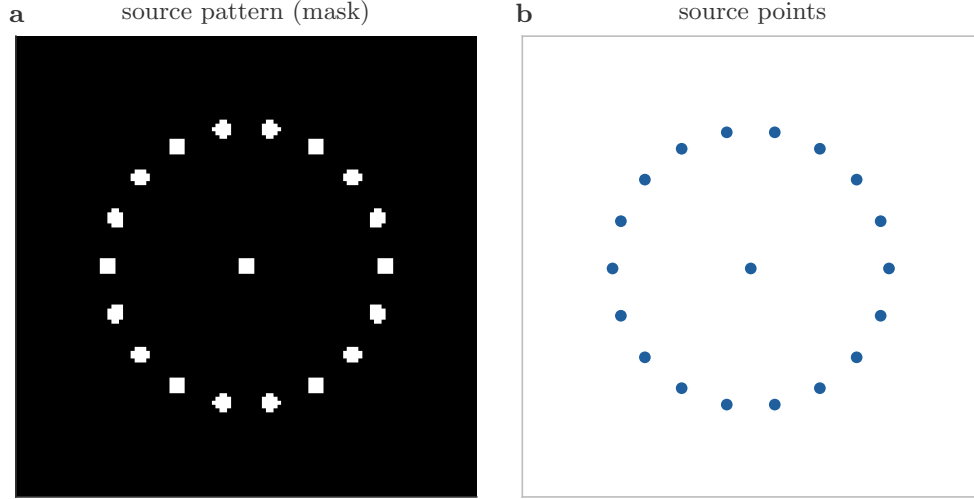


Figure 4: A source pattern (a) becomes a set of source points (b). The Virtual Experiment switches random subsets of these points on and off to build the blinking video.

| Control                       | What it does                                                                                                                          | Default          |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|------------------|
| <b>Frames</b>                 | Number of frames in the generated video (1–10000). More frames give a denser reconstruction but take longer.                          | 1000             |
| <b>Max blinkers/frame</b>     | The largest number of source points allowed to be on in any one frame. The actual number may be lower if spacing rules cannot be met. | 30               |
| <b>Blinker spacing</b>        | How far apart simultaneously-on blinkers must be: <b>separated</b> , <b>touching</b> , <b>overlapping</b> , or <b>random</b> .        | <b>separated</b> |
| <b>Manual min distance px</b> | A reference separation distance, in pixels, that the spacing mode scales.                                                             | 100.0            |
| <b>Effective min distance</b> | Read-only. Shows the separation actually applied.                                                                                     | —                |

Table 2: Controls on the **2 Blinking** page.

### 6.3 Step 3 — Image Formation

This page sets how each source point is rendered into a camera-like image: how blurred, how bright, and how noisy. Figure 5 shows one resulting frame.

| Control                     | What it does                                                                                                                 | Default |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------|---------|
| <b>sigma_px</b>             | The blur width $\sigma$ of each spot, in pixels — the point-spread width (0.5–50). Larger values give wider, blurrier spots. | 10.0    |
| <b>Spacing px</b>           | The distance, in pixels, between neighbouring points of the source mask when they are placed in the image (1–100).           | 2.6     |
| <b>Brightness</b>           | The base number of signal photons per blinker (1–1 000 000).                                                                 | 80000   |
| <b>Effective brightness</b> | Read-only. The brightness actually used (see the note below).                                                                | —       |
| <b>Background</b>           | A constant background level added to every pixel.                                                                            | 1.0     |
| <b>Read noise</b>           | The strength of the camera-like random read noise.                                                                           | 0.2     |

Table 3: Controls on the **3 Image Formation** page.

ClassRoomSTORM scales the **Brightness** you enter by a factor that grows with the number of blinkers per frame, so that busy frames stay visible. The scaled value is shown in the read-only **Effective brightness** field, and *that* value is what is used to render the frames. If your frames look brighter or dimmer than expected, check **Effective brightness**, not **Brightness**.

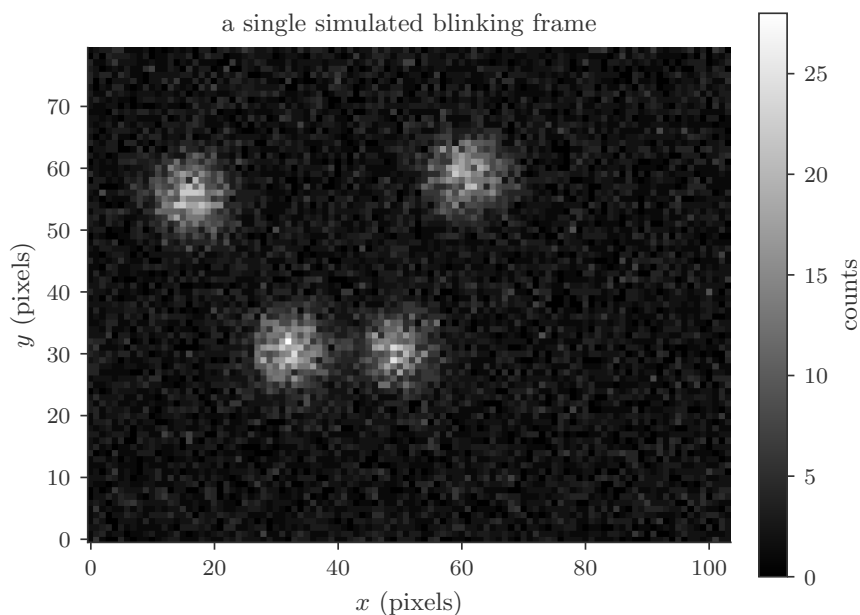


Figure 5: A single simulated blinking frame: only a few emitters are on, each appearing as a blurred spot on a noisy background.

### 6.4 Step 4 — Preview

Two buttons let you check your settings before generating anything: **Preview Source Mask** shows the pattern of source points, and **Preview Sample Blink Frame** shows one example frame. Previewing does not write any files.

## 6.5 Step 5 — Export

Choose an output folder (or accept the suggested one) and click **Export Dataset**. ClassroomSTORM writes the files listed in table 4 and then shows them in the result browser.

The **Frame backend** setting controls how virtual frames are rendered. **Auto CPU** is the recommended default. **CPU parallel** uses CPU workers, and **GPU CUDA** is experimental and requires PyTorch CUDA from `requirements-gpu.txt`. If CUDA is unavailable, ClassroomSTORM falls back to CPU generation and records the reason in `metadata.json`.

The **CPU workers (0=auto)** setting controls CPU frame rendering. Use 0 to let ClassroomSTORM use up to about 90% of available CPU workers, 1 for serial generation, or a larger value to limit CPU-parallel generation manually.

| File                            | Contents                                                                                                    |
|---------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>blinking_video.mp4</code> | The blinking video — the input for reconstruction.                                                          |
| <code>truth.csv</code>          | The exact source positions used. For comparison <i>after</i> reconstruction only (see the truth-data rule). |
| <code>preview_frame.png</code>  | A sample blinking frame.                                                                                    |
| <code>pattern_mask.png</code>   | The source-point mask.                                                                                      |
| <code>metadata.json</code>      | All generation settings.                                                                                    |

Table 4: Files written by the Virtual Experiment.

## 7 SR Reconstruction Step by Step

SR Reconstruction turns a blinking video into a reconstructed image. The video may be one you made with the Virtual Experiment or a real recorded blinking video. The window has six numbered steps: **1 Load Video**, **2 Calibration**, **3 Preprocess**, **4 Blinker Mode**, **5 Processing**, and **6 Results**.

### 7.1 Step 1 — Load Video

Click **Browse Video** and choose an MP4, AVI, or MOV file. ClassroomSTORM reads the video and shows its first frame and basic information (size, frame count, frame rate). You may also set an **Output label** and an **Output** folder for the results. The **Inspect** button repeats this inspection at any time.

The **Play video** button plays the loaded video frame by frame in the preview, and the slider beside it scrubs through the frames — a quick way to see the raw blinking data before reconstructing it.

### 7.2 Step 2 — Calibration

The **Calibration** page lets you enter a physical scale (units per pixel). In V1.1, reconstruction runs entirely in *pixel units*: the values on this page are recorded but are not applied to the results. You can safely leave this page on its default **Pixels only** setting. Working in pixels is intentional — it keeps the method general and avoids unit-conversion errors.

### 7.3 Step 3 — Preprocess

If you only want to reconstruct part of the field of view, tick **Enable crop** and set **Crop x**, **Crop y**, **Crop width**, and **Crop height**. A rectangle appears on the first-frame preview; **Show crop overlay** and **Preview cropped region** let you check it.

**Crop width** and **Crop height** start at 0. If you enable cropping but leave them at zero, reconstruction stops with an error. Inspecting a video fills these boxes with the full video size automatically; adjust them from there.

The **Reject bad frames** and **Background correction** checkboxes are shown on the **3 Preprocess** page but have no effect in V1.1. They are reserved for a future version.

## 7.4 Step 4 — Blinker Mode

The **Blinker mode** control tells ClassroomSTORM how many bright spots to expect in each frame:

**single** One localization per frame — the single brightest region.

**multiple** Several localizations per frame — each separated bright region is located independently.

**auto** ClassroomSTORM scans the first part of the video and chooses **single** or **multiple** for you. This is the default.

Use **single** for data with one blinker at a time, and **multiple** when several well-separated blinkers can be on together. If you are unsure, leave it on **auto**. The mode actually used is recorded in `summary.json`.

## 7.5 Step 5 — Processing

The **Processing backend** controls how the computation is distributed.

| Backend                          | What it does                                                                                               |
|----------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Serial</b>                    | Processes frames one at a time on the CPU. Simplest and most predictable; the recommended starting choice. |
| <b>Auto CPU</b>                  | Uses several CPU cores in parallel when that is worthwhile, capped at about 90% of available CPUs.         |
| <b>Only GPU</b>                  | Uses an NVIDIA CUDA GPU for the single-blinker path.                                                       |
| <b>Auto CPU+GPU</b>              | Uses the GPU for single blinkers and parallel CPU for multiple blinkers.                                   |
| <b>Auto CPU+GPU acceleration</b> | An experimental hybrid path.                                                                               |

Table 5: Processing-backend options.

**Workers (0=auto)** sets the number of CPU processes; 0 lets ClassroomSTORM choose up to about 90% of available CPUs. **Max frames (0=all)** limits how many frames are processed — useful for a quick test run.

The GPU backends need an NVIDIA card with CUDA and the optional GPU packages installed. If these are missing, ClassroomSTORM does *not* stop — it quietly runs on the CPU and records the fallback in `summary.json`. If GPU speed matters, open `summary.json` after the run and confirm the GPU was actually used.

## 7.6 Step 6 — Run and Results

On the **6 Results** page you may enter **Cumulative frames** (frame counts at which to save intermediate reconstructions — comma-separated whole numbers) and tick **Compare with truth after run** if a `truth.csv` is available. Click **Run Reconstruction**. When it finishes, the results appear in the browser, grouped as **Main Results**, **Diagnostics**, **Learning Progress**, **Teaching Pipeline**, and **Data Files**.

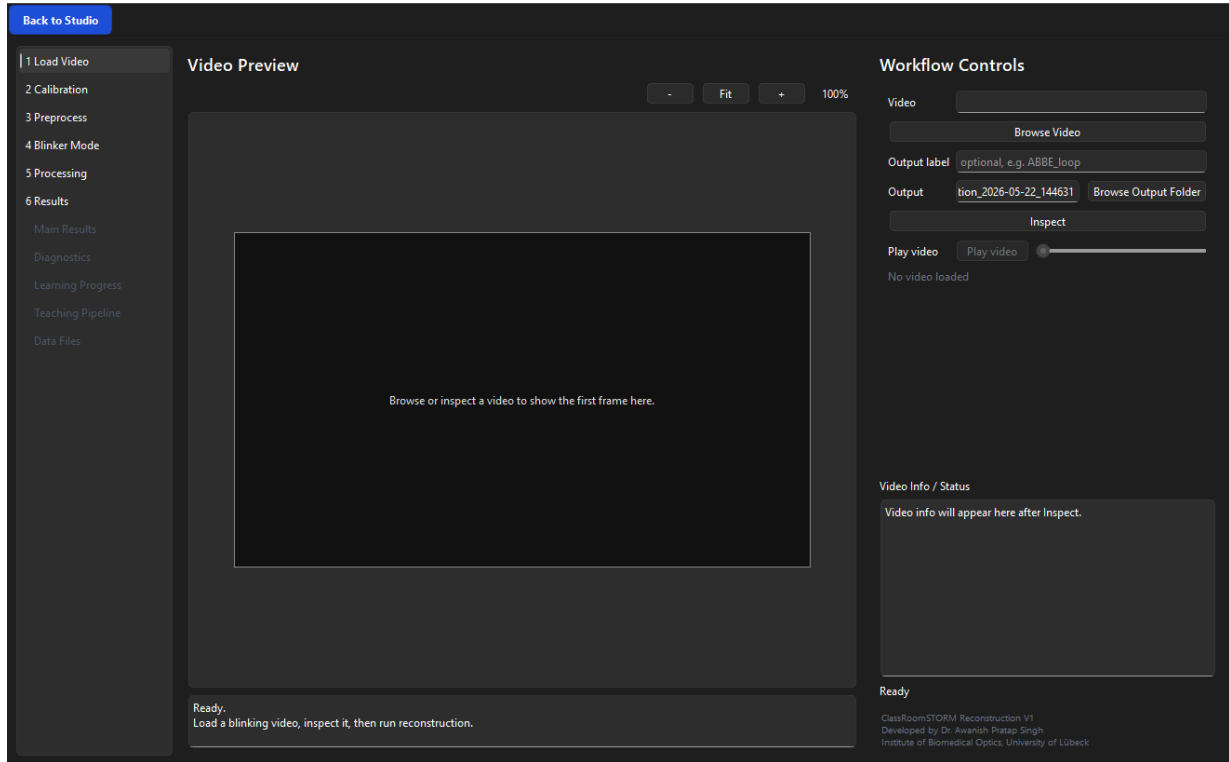


Figure 6: The SR Reconstruction window on the **1 Load Video** page, showing the numbered step list, the video preview, and the workflow controls.

## 8 Linked Profile Viewer

The **Linked Profiles** tab, in the **Main Results** group, is the clearest way to see what reconstruction achieved. It appears when a reconstruction has produced both raw and reconstructed image data.

The viewer shows the **Raw mean** image and the **Reconstruction** image side by side, with two line-profile plots beneath them. **Click anywhere on either image** to place a crosshair; the plots update to show the intensity along the row and the column through that point.

- The **red** curve is the **Raw mean** image — a wide, blurred bump.
- The **blue** curve is the **Reconstruction** — a much narrower peak at the same place.

The horizontal axis is the pixel coordinate; the vertical axis is normalized intensity (each curve scaled to a 0–1 range so the shapes can be compared directly). A narrow blue peak beside a wide red bump is the visual signature of super-resolution.

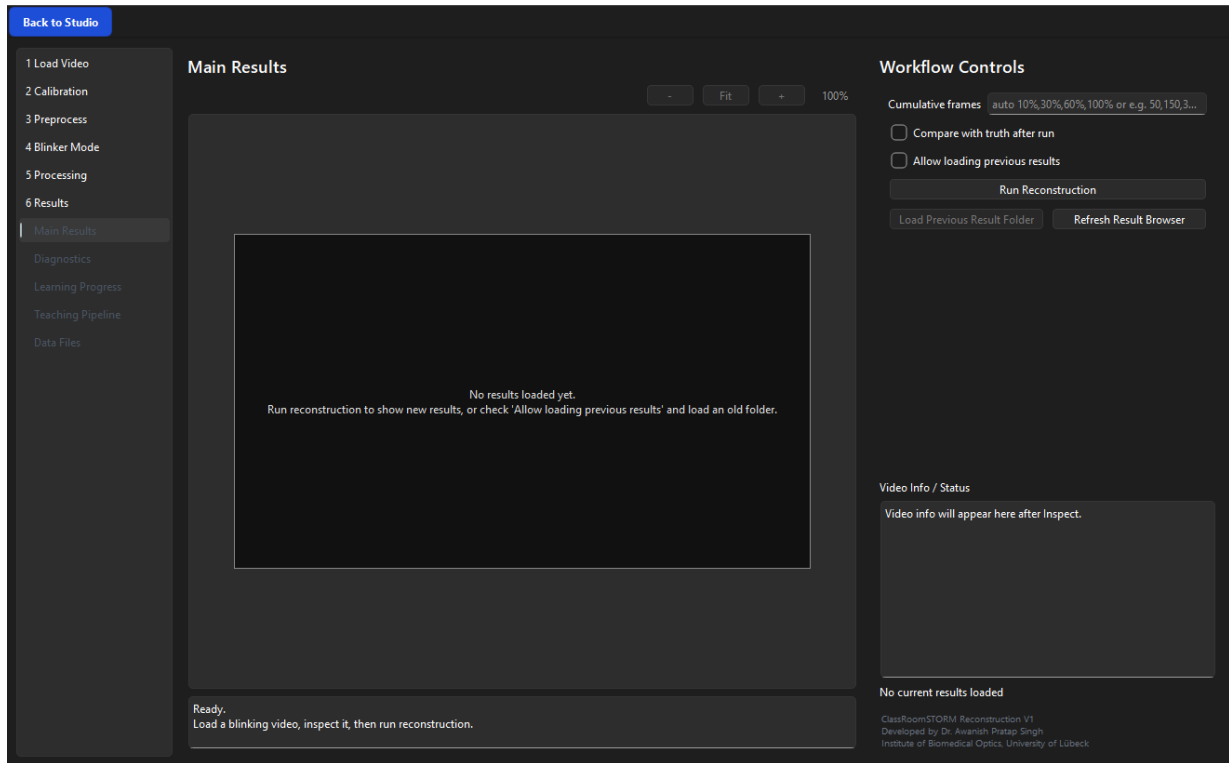


Figure 7: The SR Reconstruction **Main Results** page. The result browser fills with images, plots, and data files once a reconstruction has run.

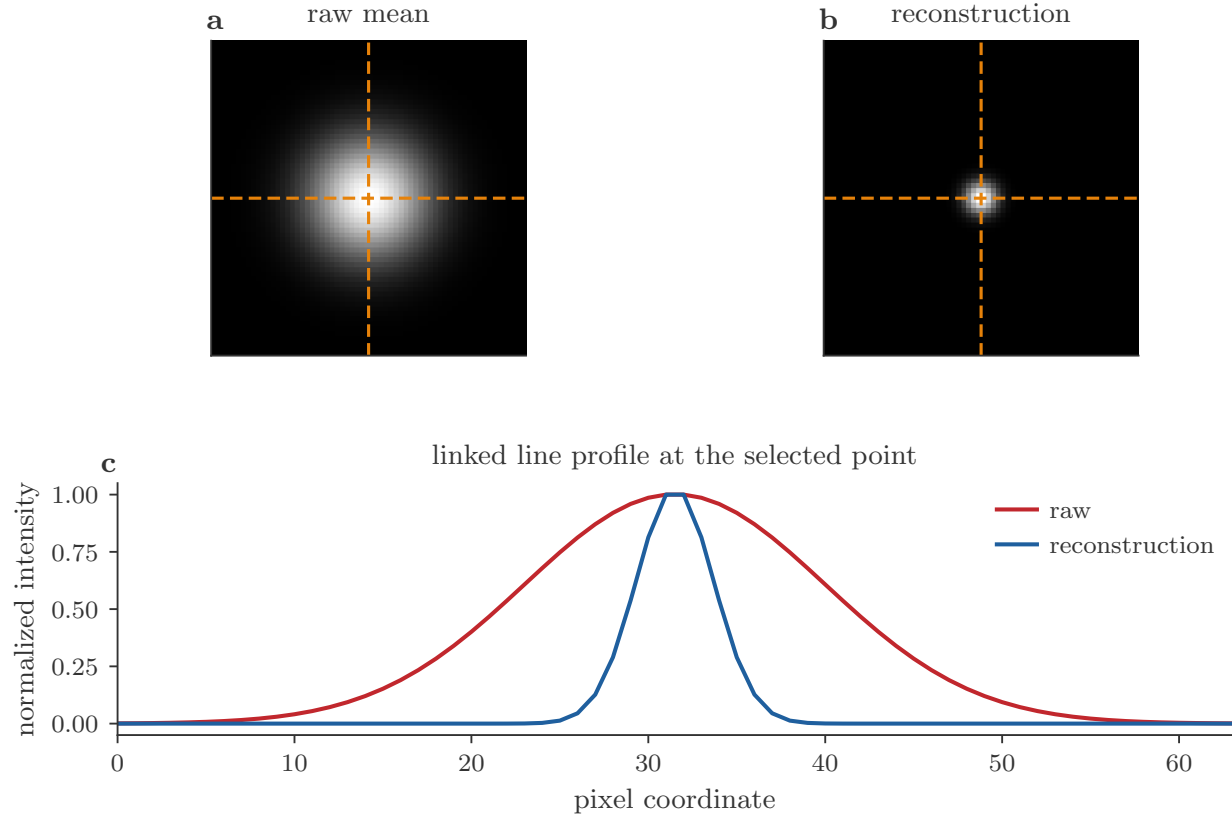


Figure 8: The linked profile viewer compares the raw and reconstructed images along the same line. Red is the raw (diffraction-limited) image; blue is the reconstruction.

## 9 Reading Output Files and Diagnostic Figures

### Raw Mean

The plain average of all processed frames — a diffraction-limited, blurry image.

### Super-Resolution

The reconstructed image, built by accumulating all localizations.

### Raw vs Super-Resolution

The two images side by side.

### Localization Scatter / Density Map / Raw Overlay

Diagnostic plots: where localizations landed, how densely, and how they sit on top of the raw image.

### Learning Progress

Cumulative reconstructions showing how the image sharpens as more frames are added.

### Teaching Pipeline

A step-by-step report (`pipeline_report .html`) and pipeline diagrams.

### Data Files

The numerical results: `localizations.csv` and `summary.json`.

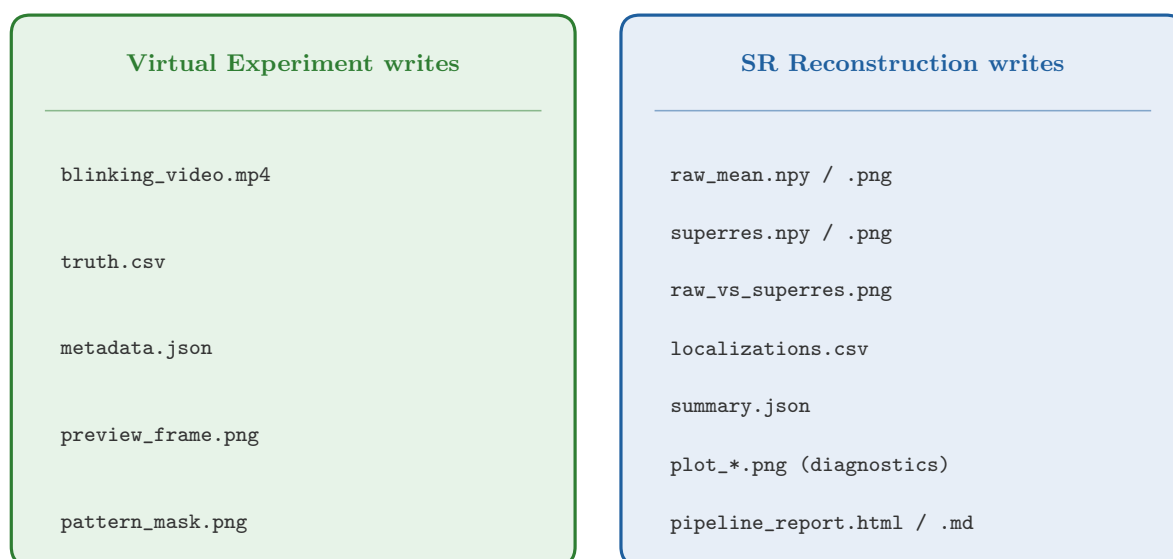


Figure 9: The files written by each workflow. Reconstruction writes both a `.npy` numerical array and a `.png` image for the raw mean and the super-resolution image; the linked profile viewer uses the `.npy` files.

## 10 Input and Output File Table

## 11 Troubleshooting

### Setup fails to download the packages.

This usually means there is no internet access, or a restricted or proxy network where Python

| Workflow           | Direction | Files                                                                                                                                                                                                                                                                                         |
|--------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Virtual Experiment | reads     | a logo image (logo mode only)                                                                                                                                                                                                                                                                 |
| Virtual Experiment | writes    | blinking_video.mp4, truth.csv, preview_frame.png, pattern_mask.png, metadata.json                                                                                                                                                                                                             |
| SR Reconstruction  | reads     | a blinking video (MP4/AVI/MOV); optionally truth.csv for comparison                                                                                                                                                                                                                           |
| SR Reconstruction  | writes    | localizations.csv; raw_mean.npy and raw_mean.png; superres.npy and superres.png; raw_vs_superres.png; localization_scatter.png; plot_density_map.png; plot_raw_localization_overlay.png; plot_cumulative_reconstruction.png; summary.json; pipeline_report.md / .html; pipeline diagram files |

Table 6: Files read and written by each workflow.

cannot verify PyPI security certificates (a `CERTIFICATE_VERIFY_FAILED` error). Try an unrestricted network, ask your IT support how pip should reach PyPI, or use a package that includes an offline `wheels` folder.

**Windows says Python was not found or opens the Microsoft Store.**

Run `setup_windows.bat` again and use its Python install option, or install Python 3.11, 3.12, or 3.13 from <https://www.python.org/>. If Windows keeps opening the Microsoft Store, disable the `python.exe` and `python3.exe` shortcuts in **Settings > Apps > Advanced app settings > App execution aliases**.

**The application will not start, or a “ModuleNotFoundError” appears.**

The required packages are not installed. Run the setup script (`setup_windows.bat` or `setup_macos.command`) and wait for the *Setup completed* message before launching again.

**The launcher disappeared after I clicked a workflow.**

This is expected. The launcher closes itself when a workflow opens. Use the **Back to Studio** toolbar button at the top of the workflow window, or run the launch command again to get it back.

**No bright spots are detected / the reconstruction is empty.**

Check the video contrast on the first frame. Very dark or very noisy videos may have no clear spots. Try **multiple** blinker mode.

**The reconstruction looks too sparse.**

Use more frames, or (for a Virtual Experiment) increase **Max blinkers/frame**.

**A GPU backend seems no faster.**

It probably fell back to the CPU. Check `summary.json`; install the optional GPU packages if you have an NVIDIA CUDA card.

**Reconstruction fails immediately with a crop error.**

**Crop width/Crop height** are zero. Inspect the video first to fill them, or untick **Enable crop**.

**The video will not load.**

Make sure it is an MP4, AVI, or MOV file that OpenCV can read.

**The Linked Profiles tab is missing.**

It appears only when a reconstruction has produced the raw and super-resolution data. Run a reconstruction first.

**Truth comparison does nothing.**

Make sure `truth.csv` is in the same folder as the video and that **Compare with truth after run** is ticked.

**A run is very slow.**

Lower **Max frames** for a test, or use **Auto CPU**.

## 12 Glossary

### **Blinker**

A light source that switches on and off; it appears as a bright spot when on.

### **Localization**

An estimate of the centre position of one bright spot.

### **Centroid**

The brightness-weighted average position of a spot's pixels; the method ClassroomSTORM uses to localize.

### **Threshold**

The brightness level above which pixels are treated as part of a spot.

### **Point-spread width ( $\sigma$ )**

How wide a single point of light appears after blurring; the `sigma_px` control.

### **Reconstructed (super-resolution) image**

The sharp image built by accumulating localizations.

### **Raw mean**

The blurry plain average of the video frames.

### **Ground truth**

The known, exact source positions, saved in `truth.csv`, used only to score a reconstruction afterwards.

### **Frame**

One image in the video.

### **Signal photons**

The brightness of a blinker, set by **Brightness**.

### **Background**

A constant light level added to every pixel.

### **Read noise**

Random camera-like noise added to each pixel.

### **Field of view / crop**

The region of the video that is processed.

### **Pipeline report**

The step-by-step teaching report produced after a run.

## 13 Instructor Notes

**Short session (2–3 hours).** Use the Virtual Experiment only. Students generate a dataset, reconstruct it, and explore the linked profiles. A good guided question: *why* is the blue curve narrower than the red one?

**Longer project (1–2 weeks).** Add a real recorded blinking video — for example from a classroom optical setup or an LED source — and have students compare reconstructions of simulated and real data.

**Learning goals.**

- Compare a raw (diffraction-limited) image with a localization-based reconstruction.
- Understand the truth-data rule: why the answer key is hidden until after reconstruction.
- Explore how blur width, noise, blinker density, and overlap affect the result.
- Treat physical calibration as optional — the core workflow is pixel-first.

**Suggested parameter sweeps.** Have students vary one setting at a time and predict the effect before running: **sigma\_px** (blur), **Read noise**, **Max blinkers/frame** (density and overlap), and **Frames** (sampling). The companion *Theory* document gives the physics behind each of these.